APPLIANCE ENERGY PREDICTION USING DEEP LEARNING

Technical Assessment Report: Forecasting Household Appliance Energy Consumption

DHANUSHKA RATHNAYAKE
15 OCT 2026

TABLE OF CONTENTS

INTRODUCTION 1
a) Objectives 1
b) Approach and headline results 1

DATA INSIGHTS 2
a) The data set 2
b) Target variable and temporal patterns 2
c) Correlations and relevance of the sensors 3
d) Key observations 3

PREPROCESSING 4
a) Data cleaning and missing values 4
b) Outlier detection and treatment 4
c) Scaling and normalization 4
d) Chronological splitting 5
e) Leakage-free formulation 5

FEATURE ENGINEERING 5
a) Features created 5
b) Lag selection from the autocorrelation analysis 5
c) Feature selection 6

MODEL DESIGN 7
a) Inputs and baselines 7
b) Candidate architectures 8
c) Activation functions, optimizers and loss functions 8
d) Layers, neurons, look-back and regularization 9

RESULTS 9
a) Evaluation metrics 9
b) Comparison with the baseline models 9
c) Training behaviour 10
d) Predicted versus actual values and residuals 11

MODEL OPTIMIZATION 12
a) Scope and a reliable way to rank configurations 12
b) Hyper-parameter tuning 12
c) Regularization: dropout layers and dropout rates 13
d) Early stopping 13
e) Final optimized models and seed ensemble 14
f) Improvement in performance metrics 14

CHALLENGES AND SOLUTIONS 16

CONCLUSION 17
Potential areas for future work 17

REFERENCES 17

INTRODUCTION

Household energy use is a growing share of total electricity demand, and the electricity consumed by appliances and lights is among its most variable parts. Accurate short-term forecasts of this consumption allow smart-home controllers, demand-response programmes and energy-management systems to schedule loads, avoid peaks and detect abnormal behaviour. When the forecast must run close to the meter, on a small edge device, the model also has to be light enough to be trained and executed with little memory and computation.

This report documents the design, training and optimization of deep-learning models that forecast the energy consumption of the appliances in a house. The task is **one-step-ahead forecasting**: given everything known up to now, predict the appliance energy consumption (in Wh) of the next 10-minute interval. The work uses a public data set [1] of about 4.5 months of 10-minute measurements of the appliance energy consumption together with indoor and outdoor conditions.

a) Objectives

- analyse the data set and prepare it for modelling (exploratory analysis, missing values, outliers, scaling, chronological splitting);
- create and select features that improve forecasting accuracy, and justify each of them;
- design deep-learning models (LSTM, GRU, CNN-LSTM and MLP), choose their layers, activation functions, optimizers and loss function, and compare them with simple baseline models;
- train and evaluate the models with MAE, RMSE, MAPE and R^2, and visualize the predictions and errors;
- optimize the selected models through hyper-parameter tuning, dropout and early stopping, and report the improvement.

b) Approach and headline results

All experiments follow one strict protocol. The first 80 % of the time series is used for training and the last 20 % for testing; a validation set is taken from the training part only, and the test set is used only for the final comparison. Every metric is computed in Wh. The reference model, *persistence* (predict the previous interval), reaches an RMSE of 66.4 Wh on the test set. After feature selection, model design and optimization, the best deep-learning model, an ensemble of three single-layer LSTM networks, reaches 56.9 Wh ($R^2 = 0.609$) and is significantly better than persistence, linear regression and a random forest (RMSE 58.4 Wh). A single-hidden-layer MLP with only 11 thousand parameters is almost as accurate. The report also shows, with evidence, where the gains are small and cannot be separated from random variation.

Code availability. The complete analysis is contained in the Jupyter notebook `notebooks/Appliance_Energy_Prediction.ipynb`. The repository github.com/Dhanushka0626/Appliance-Energy-Prediction also contains the reusable code in `src/` (preprocessing, feature engineering, model definition and a training script `train.py` that rebuilds the final model from the raw data), the raw and processed data in `data/`, and the trained model with its scalers in `models/`. The experiments used Python, pandas, scikit-learn [13] and TensorFlow/Keras [14]; all random seeds are fixed, so the results are reproducible.

DATA INSIGHTS

a) The data set

The data set contains 19,735 observations recorded every 10 minutes between 11 January 2016 17:00 and 27 May 2016 18:00 (Table 1). The target, `Appliances`, is the energy used by appliances in Wh during the 10-minute interval. The other columns are the lighting energy, temperature and humidity readings (T1–T9, RH_1–RH_9), six outdoor weather variables, and two further columns, `rv1` and `rv2`. The data set was obtained from a public repository [1].

Table 1: Summary of the data set.

Property	Value
Source	public data set, UCI Machine Learning Repository [1]
Observations	19,735 (regular 10-minute interval, 29 columns)
Period	11 Jan 2016 17:00 to 27 May 2016 18:00 (about 4.5 months)
Target	Appliances (Wh per 10 minutes): minimum 10, median 60, mean 97.7, standard deviation 102.5, maximum 1,080
Inputs	lights; temperatures T1–T9 and humidities RH_1–RH_9; weather variables T_out, RH_out, Press_mm_hg, Windspeed, Visibility, Tdewpoint; columns rv1, rv2

b) Target variable and temporal patterns

The target is strongly right-skewed (Figure 1): half of all intervals use 60 Wh or less, while a small number of intervals exceed several hundred Wh. The time series shows no long-term trend (the mean is about 97 Wh in every month of the training period) but it is very spiky, with short bursts that are presumably caused by appliances being switched on.

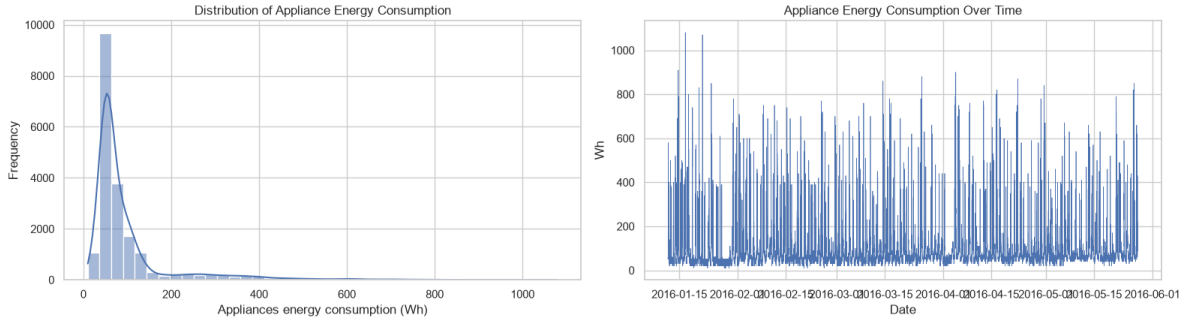


Figure 1: Distribution of the appliance consumption (left) and its time series (right).

Consumption follows a clear daily cycle (Figure 2). It is lowest at night (about 48 Wh at 03:00), rises in the morning, shows a plateau around midday and peaks in the evening (about 190 Wh at 18:00). The day-of-week averages are similar (lowest on Tuesday, about 87 Wh; highest on Monday, about 112 Wh), and the hour-by-weekday heat map shows the same evening peak on every day. Hour of the day is therefore an important feature, while the day of the week is weak (Figures 2 and 3).

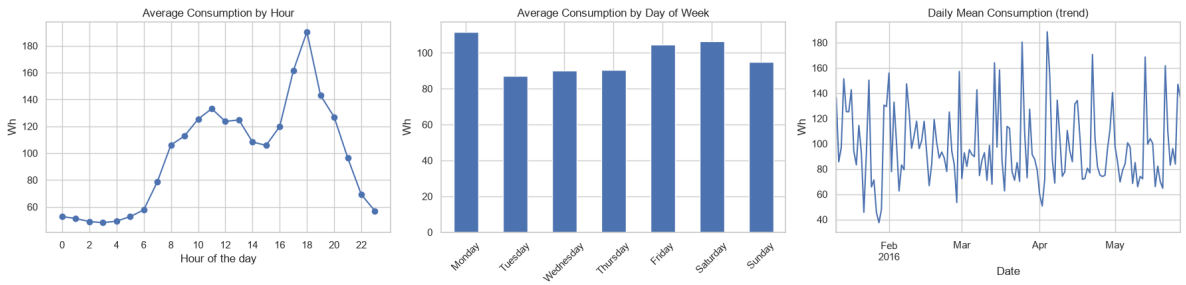


Figure 2: Average consumption by hour of the day (left), by day of the week (centre) and the daily mean over time (right).

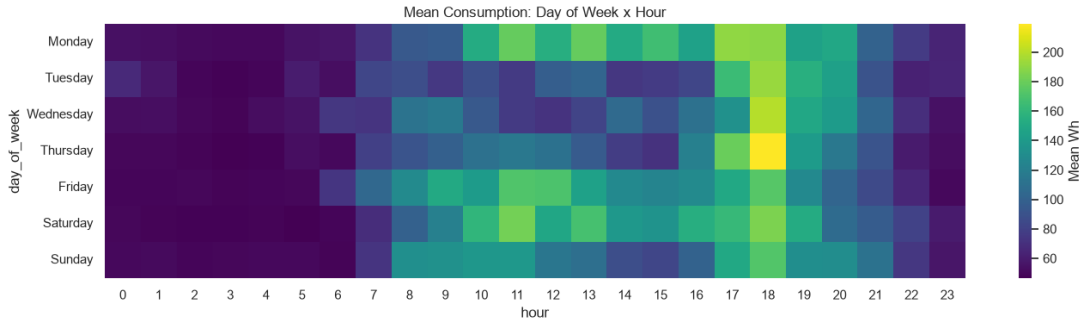


Figure 3: Mean consumption by day of the week and hour of the day.

c) Correlations and relevance of the sensors

All linear correlations between the original columns and the target are weak (Figure 4): the largest are lighting energy ($r = +0.20$), outdoor humidity ($r = -0.15$) and the temperatures T2 and T6 ($r \approx +0.12$). The temperature and humidity columns are strongly correlated with each other, so they carry largely redundant information. The columns `rv1` and `rv2` are identical to each other and uncorrelated with the target ($r = -0.01$), so they carry no information. This already suggests that the recent history of the consumption itself will be more informative than the household sensors.

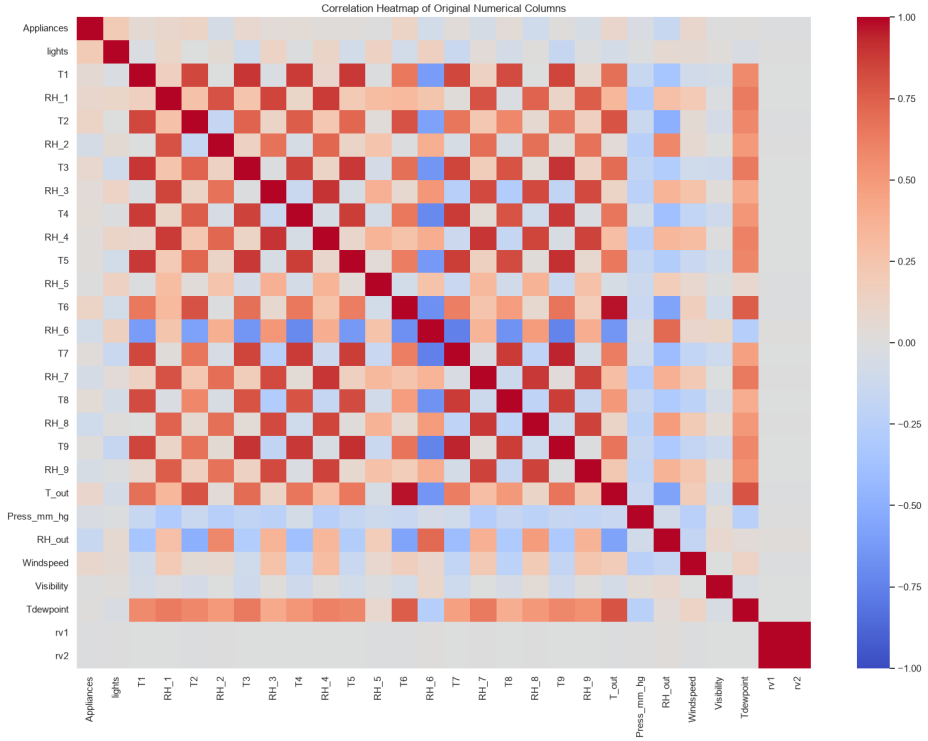


Figure 4: Correlation heat map of the original numerical columns.

d) Key observations

- The data are complete and regular: no missing values, duplicates, invalid or repeated timestamps and no negative values.
- The target is skewed and spiky; errors will be dominated by unpredictable bursts, so RMSE and MAE will tell different stories.
- A daily cycle exists (evening peak), but the weekly cycle is weak.
- The measurement period covers only winter-to-late-spring conditions at a single site; the weather changes systematically over the period, which becomes important for feature selection (Section on Feature Engineering).

PREPROCESSING

a) Data cleaning and missing values

The timestamps were converted to datetime format and sorted chronologically. The quality checks found no invalid timestamps, no repeated timestamps, no duplicate rows, no missing values and no negative consumption. All 19,734 differences between consecutive timestamps equal exactly 10 minutes. No imputation was therefore necessary. The notebook nevertheless contains the documented strategy for new data: short gaps in the *sensor* columns are filled by time-based linear interpolation (the signals change smoothly over minutes), rows with an invalid timestamp or a missing target are removed, and the target itself is never imputed because that would invent consumption.

b) Outlier detection and treatment

Outliers were detected on the training period only, using the inter-quartile range (IQR) rule, the Z-score rule and box plots (Figure 5). With Q1 = 50 Wh, Q3 = 100 Wh and an upper bound of 175 Wh, the IQR rule flags 1,776 training observations (11.25 %); the Z-score rule ($|z| > 3$) flags 416 (2.63 %). The largest value (1,080 Wh) is physically plausible.

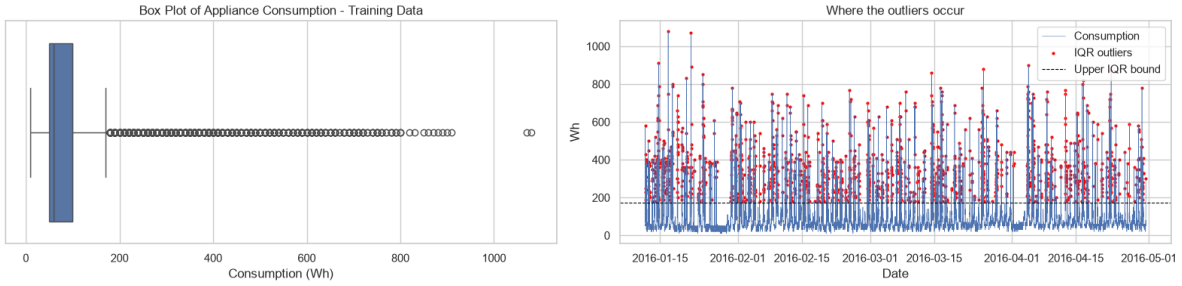


Figure 5: Box plot of the training target (left) and the location of the IQR outliers over time (right).

The high values are genuine appliance activity, not measurement errors, and they are exactly the events a forecaster should anticipate. Instead of deciding by belief, the effect of treating them was measured: the training target was capped at the IQR upper bound (175 Wh) and the models were evaluated on the untouched validation targets (Table 2). Capping made the RMSE much worse, and the R^2 fell from about 0.58–0.65 to 0.32–0.36; it only improved the MAPE, which is a percentage dominated by the many small values. **The outliers were therefore retained**.

Table 2: Effect of capping the training target on validation accuracy.

Training target	Model	MAE (Wh)	RMSE (Wh)	R^2
Outliers retained	Linear Regression	27.87	62.78	0.578
Outliers retained	Random Forest	26.19	57.58	0.645
Capped at 175 Wh	Linear Regression	31.92	79.73	0.319
Capped at 175 Wh	Random Forest	29.58	77.52	0.356

c) Scaling and normalization

Inputs have very different units and ranges (temperatures in °C, humidity in %, visibility in metres, lags in Wh), and neural networks and linear models train poorly on unscaled inputs. **Standardization** (zero mean, unit variance) was chosen over Min-Max scaling because the target and several features are right-skewed with large peaks: Min-Max scaling would squeeze most values into a very small range, and unseen peaks in the test period would fall outside [0, 1]. The scalers (inputs and target) are **fitted on the training sub-set only** and then applied to the validation and test periods, so no information from the future leaks into training. The network output is converted back to Wh before any metric is calculated.

d) Chronological splitting

A time series must not be shuffled. The data were split chronologically: the **first 80 %** (15,788 observations, 11 January to 30 April) for training and the **last 20 %** (3,947 observations, 30 April to 27 May) for testing. Model selection needs held-out data, but the test set must stay untouched until the end; the **last 15 % of the training part** was therefore set aside as a validation set (Table 3). The first 144 rows (one day) cannot be used because the 24-hour lag is not available for them.

Table 3: Chronological split of the data (the first 80 % is the training part).

Part	Rows	Period	Used for
Training sub-set	13,298	12 Jan 17:00 – 14 Apr 01:10	fitting scalers and models, feature selection
Validation	2,346	14 Apr 01:20 – 30 Apr 08:10	early stopping and all design decisions
Test	3,947	30 Apr 08:20 – 27 May 18:00	final evaluation only (20 % of the data)

e) Leakage-free formulation

Because the aim is forecasting, the features of row *i* may only use information that exists *before* interval *i*: lags and rolling statistics use earlier consumption values only; the sensor and weather readings are taken from the *previous* interval (shifted by one step); calendar features describe interval *i* itself and are known in advance. An automatic test perturbs every raw value of one interval and verifies that the features of that interval do not change, while those of the next interval do. This test passes.

FEATURE ENGINEERING

a) Features created

Features were created in five groups and then selected with evidence (Table 4). Every group is motivated by the data analysis: the daily cycle (time features), the strong short-term persistence (lags and rolling statistics), and the physical relation between weather and energy demand (interaction and domain features).

Table 4: Engineered feature groups and their motivation.

Group	Features created	Why it should help
Time-based	hour of the day (sin/cos with minute resolution), day of week (sin/cos), month (sin/cos), weekday/weekend indicator	daily cycle with an evening peak; cyclical encoding keeps 23:50 and 00:00 close together
Lags	consumption 10, 20, 30 min, 1, 2, 3 h and 24 h earlier	consumption is strongly autocorrelated (Figure 6)
Rolling windows	mean over 1 h, 3 h, 24 h; standard deviation over 1 h, 3 h (past values only)	smooth the spiky signal; describe the recent level and volatility
Interactions	T2 × RH_2 and T_out × RH_out	temperature and humidity act together on heating and comfort
Domain-specific	public-holiday flag, mean indoor temperature and humidity, indoor-outdoor temperature difference, lights-on indicator	heating demand depends on the temperature difference; holidays change occupancy

b) Lag selection from the autocorrelation analysis

The lags were not guessed but derived from the autocorrelation function (ACF) of the training target (Figure 6, Table 5). The ACF decays quickly over the first hour (0.76 at 10 minutes, 0.54 at 20 minutes, 0.44 at 30 minutes), keeps a secondary bump around lag 10 (about 100 minutes, a typical usage cycle) and rises again at lag 144, exactly 24 hours, which shows that the consumption repeats from day to day. All selected lags are statistically significant.

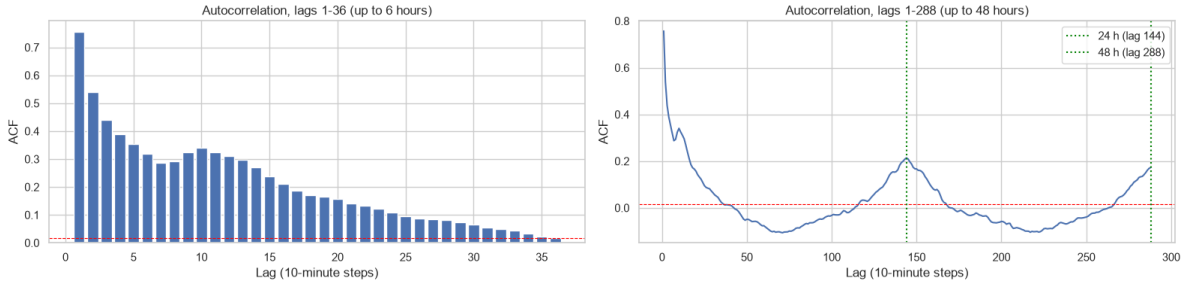


Figure 6: Autocorrelation of the training target up to 6 hours (left) and 48 hours (right); dashed red line: approximate 95 % band.

Table 5: Selected lags and their autocorrelation in the training period.

Lag (steps)	**1**	**2**	**3**	**6**	**12**	**18**	**144**
Time before the target	10 min	20 min	30 min	1 h	2 h	3 h	24 h
Autocorrelation	0.757	0.540	0.439	0.320	0.311	0.172	0.214

c) Feature selection

Selection used the training period only. Three independent methods were combined by voting: (1) the absolute correlation with the target ($|r| \geq 0.05$); (2) the importance in a random forest [7] (features that make up 95 % of the cumulative importance); and (3) recursive feature elimination (RFE) [8] with a ridge regression, down to 25 features. A feature is kept when at least two of the three methods select it. Duplicated columns are removed first (`rv2` is identical to `rv1`); the remaining random column `rv1` received only one vote. In total, 33 of the 52 candidates received at least two votes. The lag of 10 minutes alone carries 44 % of the random-forest importance (Figure 7), followed by the 1-hour rolling statistics and the lags of 20 and 30 minutes.

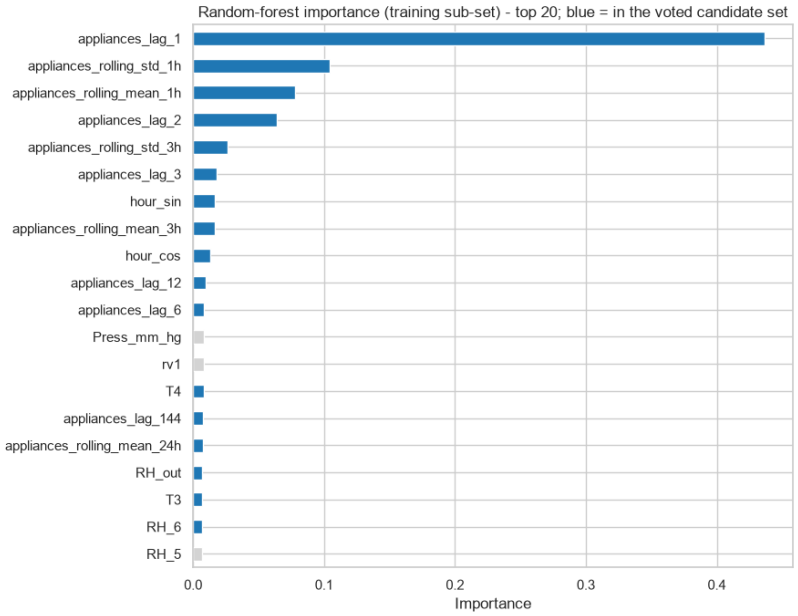


Figure 7: Random-forest feature importance on the training sub-set (top 20); blue bars are in the voted set.

Robustness to seasonal drift. The weather changes systematically over the measurement period: the mean outdoor temperature is about 6 °C in the training weeks and about 14 °C in the test weeks. A model that relies on the absolute value of a temperature can look good on a validation block from the same season and then fail later. A single validation block cannot reveal this. The candidate feature sets were therefore compared with **walk-forward (expanding-window) validation inside the training part**: five consecutive folds, each trained on everything before it and tested on the next block (Table 6). Removing

the raw sensor and weather columns gave the lowest error for the random forest and for ridge regression alike, so the raw indoor and outdoor measurements do not improve the forecast and make it less stable as the season changes. (This test was added after a first run showed a sensor-based random forest degrading on the warmer test weeks; the test itself and the selection use training data only.)

Table 6: Walk-forward feature-group test (mean RMSE over five folds, training part only).

Feature set	# features	Random forest RMSE (Wh)	Ridge RMSE (Wh)
Voted set	33	65.54	66.75
Voted set without outdoor weather	26	65.19	66.57
Voted set without any environment column	14	63.85	66.30
All past-consumption and calendar features	20	63.64	66.38

The set *voted, without environment columns* was chosen: it is within 1 % of the best error and has 14 instead of 20 features. The final features are the lags 1, 2, 3, 6, 12, 18 and 144; the rolling means over 1, 3 and 24 hours; the rolling standard deviations over 1 and 3 hours; and the sine and cosine of the time of day. The weekday, month and holiday features were dropped by the voting (0–1 votes): the weekly pattern is weak, and only four public holidays fall in the period, so there is nothing to learn from them. The interaction and indoor/outdoor domain features were dropped by the walk-forward test together with the raw sensors, for the reason given above. The justification for each group is summarized in Table 7.

Table 7: Justification of the engineered and selected features.

Feature(s)	Evidence	Decision
lag 10, 20, 30 min	ACF 0.76, 0.54, 0.44; lag 10 min has 44 % of the forest importance	kept
lag 1, 2, 3 h	ACF 0.32, 0.31, 0.17; selected by all voting methods	kept
lag 24 h, rolling mean 24 h	ACF 0.21 at 24 h: the day repeats	kept
rolling mean 1 h, 3 h	correlation 0.60 and 0.52 with the target	kept
rolling std 1 h, 3 h	correlation 0.57 and 0.53; spikes cluster in time	kept
hour of day (sin, cos)	evening peak (190 Wh) vs. night (48 Wh); three votes	kept
day of week, month, weekend, holiday	weak weekly pattern; four holidays; 0–1 votes	dropped
interactions, indoor/outdoor domain features	weak correlation; drift with the season (Table 6)	dropped
raw sensors T1–T9, RH_1–RH_9, weather, lights	weak correlation; seasonal drift; random columns got votes by chance	dropped

MODEL DESIGN

a) Inputs and baselines

The deep-learning models receive a **window of the last *L* rows** of the selected features. The window for target row i covers rows $i - L + 1$ to i; row i is included on purpose, because it holds the most recent consumption (the 10-minute lag). Simple baselines establish the benchmark that the networks must beat: *persistence* (previous interval), *seasonal persistence* (the same time on the previous day), *linear regression*, *ridge regression* and a *random forest* [7] with 200 trees. On the validation set the random forest has the lowest baseline RMSE (57.6 Wh), then linear regression (62.8 Wh) and persistence (69.9 Wh); the seasonal persistence is far worse (117.6 Wh, $R^2 < 0$) because the exact daily repetition is weak.

b) Candidate architectures

Six architectures were compared under identical conditions (same inputs, loss, optimizer and early stopping), so that the comparison isolates the architecture (Table 8). The LSTM [2] and the GRU [3] are gated recurrent networks that can keep information over a window; the GRU has fewer parameters. The CNN-LSTM first extracts short-term shapes with two one-dimensional convolutions and passes them to an LSTM. The MLP has no recurrence and acts as a check on whether recurrence is needed at all.

Table 8: Architecture comparison on the validation set (look-back 12, Adam, MSE, dropout 0.2).

Architecture	Layers	Parameters	MAE (Wh)	RMSE (Wh)	R^2
LSTM-1L	LSTM 64, dense 32	22,337	26.27	58.70	0.631
MLP	dense 128, 64, 32	32,001	27.40	59.43	0.622
LSTM-2L	LSTM 64, 32, dense 16	33,185	26.48	59.57	0.620
GRU-2L	GRU 64, 32, dense 16	25,313	26.20	59.66	0.619
CNN-LSTM	Conv 32, Conv 32, LSTM 64, dense 32	31,425	27.21	59.76	0.618
GRU-1L	GRU 64, dense 32	17,473	26.64	59.87	0.616

All six are within 2 % of each other, which is about the size of the random-seed effect. The single-layer LSTM has the lowest validation error and is the simplest recurrent model (no overfitting gap). The two-layer variants, the GRU and the CNN-LSTM are not better, and the non-sequential MLP is almost as good: once lag and rolling-window features summarize the history, recurrence adds little. A single hidden layer was therefore kept for the optimization stage, which also suits an edge device. The final networks are drawn in Figure 8.

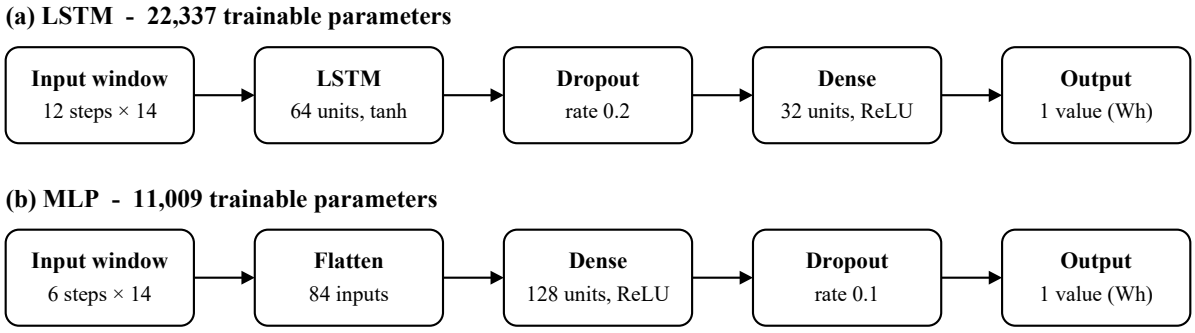


Figure 8: The two final networks: (a) LSTM with a look-back of 12 steps and (b) MLP with a look-back of 6 steps.

c) Activation functions, optimizers and loss functions

Activation functions. The recurrent cell uses *tanh*, which keeps the recurrent state bounded in [−1, 1] over the window; the dense hidden layer uses *ReLU*, which avoids saturation and vanishing gradients and is cheap to compute. The combinations tanh/ReLU, tanh/tanh and ReLU/ReLU were compared; tanh/ReLU gave the lowest validation RMSE (58.70 against 59.19 and 59.70 Wh), which agrees with the theory (Table 9). **Optimizers.** Adam [4] adapts the step size of every weight and needs little tuning; it was compared with RMSprop, Nadam and SGD with momentum. Adam, Nadam and RMSprop were practically tied (58.70, 58.83, 58.86 Wh), while SGD was clearly worse (61.55 Wh) and had not converged after 40 epochs. **Adam** was kept, with a learning rate of 0.001 and gradient clipping at norm 1.0. **Loss functions.** For regression the choice decides what is optimized: MSE penalizes large errors and is consistent with the RMSE; MAE is robust to peaks; the Huber loss [12] is quadratic for small and linear for large errors. MSE gave the best RMSE and R^2, whereas MAE and Huber lowered the MAE and MAPE but under-predicted the peaks (Table 9). Because the selection criterion of this study is RMSE, **MSE** was kept; Huber would be a reasonable alternative if the typical error mattered more than the peaks.

Table 9: Activation, optimizer and loss-function comparison (validation set).

Design choice	Option	MAE (Wh)	RMSE (Wh)	Comment
Activations (recurrent / dense)	tanh / ReLU	26.27	58.70	selected
	tanh / tanh	26.90	59.19	
	ReLU / ReLU	27.00	59.70	
Optimizer	Adam	26.27	58.70	selected
	Nadam	25.97	58.83	tied
	RMSprop	26.62	58.86	tied
	SGD + momentum	27.35	61.55	not converged
Loss function	MSE	26.27	58.70	selected (best RMSE)
	Huber	24.80	59.98	lower MAE, worse RMSE
	MAE	23.70	61.63	lowest MAE, worst RMSE

d) Layers, neurons, look-back and regularization

The number of neurons and the look-back window were compared on the validation set. Half the units (59.63 Wh) is too small and double (59.15 Wh) overfits more for no gain, so 64 units with a dense head of 32 were kept. A look-back of 12 steps (2 hours) was the best (58.70 Wh) against 6 steps (59.80 Wh) and 36 steps (59.00 Wh): longer windows do not help because the 3-hour and 24-hour behaviour is already contained in the rolling and lag features. To control overfitting, **dropout** [5] was added after the recurrent layer: without dropout the training RMSE was 54.0 Wh against 59.6 Wh on validation (a gap of 5.5 Wh), whereas a rate of 0.2 removed the gap and gave the best validation error (58.70 Wh). An L2 weight penalty did not help (58.76 and 59.16 Wh), and early stopping with the restoration of the best weights is active in every run [11]. Section *Model Optimization* refines these choices with a more reliable procedure.

RESULTS

a) Evaluation metrics

Four metrics are reported, all computed in Wh on the original scale: the mean absolute error **MAE** = $(1/n)\ \Sigma\ |y - \hat{y}|$; the root mean squared error **RMSE** = $\sqrt{((1/n)\ \Sigma\ (y - \hat{y})^2)}$, which penalizes large misses more; the mean absolute percentage error **MAPE** = $(100/n)\ \Sigma\ |y - \hat{y}| / y$ (the target is never zero in this data set); and the coefficient of determination $\mathbf{R^2} = 1 - \Sigma\ (y - \hat{y})^2 / \Sigma\ (y - \bar{y})^2$ (1 is perfect, 0 is as good as predicting the mean).

b) Comparison with the baseline models

Table 10 shows the test-set results (3,947 intervals) of all baselines and of the architectures of Table 8, with their default settings. The test set was not used for any decision. Persistence is a strong baseline for the typical error (MAE 26.65 Wh, MAPE 21.9 %, the best MAPE of all models) because the next 10 minutes resemble the last 10 minutes (lag-1 autocorrelation 0.76), but it is poor at the large changes (RMSE 66.4 Wh). Linear regression, the random forest and the networks all reduce the RMSE by 10–14 %, and the recurrent networks are slightly better than the random forest. The differences among the deep-learning models are small, of the order of the run-to-run variation (about ± 1 Wh across random seeds), so their order on the test set must not be over-interpreted.

Table 10: Test-set results of the baselines and the architectures of Table 8 (default hyper-parameters, seed 42).

Model	MAE (Wh)	RMSE (Wh)	MAPE (%)	R^2
CNN-LSTM	26.47	57.31	23.9	0.604
LSTM-1L (selected in Section 3)	25.84	57.39	23.0	0.603
GRU-1L	26.00	57.41	23.0	0.602
LSTM-2L	25.84	57.93	22.9	0.595
GRU-2L	25.54	58.03	22.7	0.594
Random Forest	26.33	58.40	23.9	0.588
MLP (two hidden layers)	27.13	59.33	25.9	0.575
Linear Regression	27.13	59.82	25.1	0.568
Ridge Regression	27.13	59.82	25.1	0.568
Persistence	26.65	66.44	21.9	0.467
Seasonal persistence (24 h)	53.68	112.18	51.1	-0.518

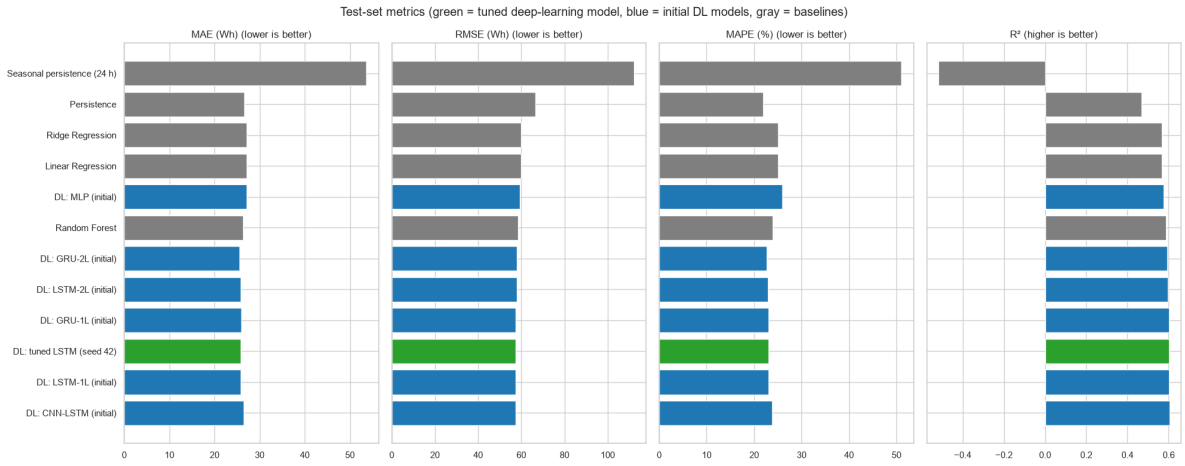


Figure 9: Test-set metrics of all models (green: tuned model, blue: deep-learning models, grey: baselines).

c) Training behaviour

Figure 10 (left) shows the training and validation loss of the selected LSTM for three random seeds: both curves fall quickly, the validation loss flattens after about 15–25 epochs and early stopping ends the training, restoring the best weights. The gap between the curves is small, so the model is not strongly overfitted. The middle panel shows the same behaviour in Wh, and the right panel compares the validation loss of the six architectures: all converge to about the same level.

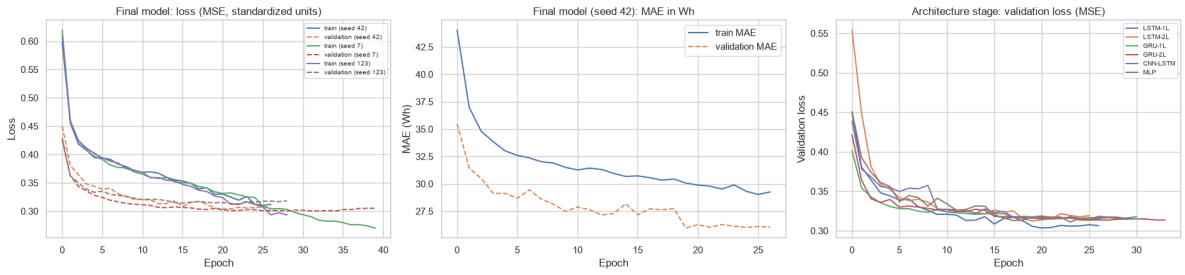


Figure 10: Training and validation loss of the selected LSTM for three seeds (left), its MAE in Wh (centre), and the validation loss of all architectures (right).

d) Predicted versus actual values and residuals

Figure 11 compares the predictions of the models with the actual consumption in the first days of the test period, and Figure 12 shows predicted against actual values. The networks follow the daily rhythm and the medium-sized bursts well, but, like every model, they under-predict the highest peaks and cannot anticipate the start of a burst: points above 400 Wh lie far below the diagonal. The residual analysis (Figure 13) shows errors that are centred on zero (mean residual about 1 Wh) with a long right tail created by the unpredictable bursts; the largest residuals reach about 600 Wh for every model, including persistence.

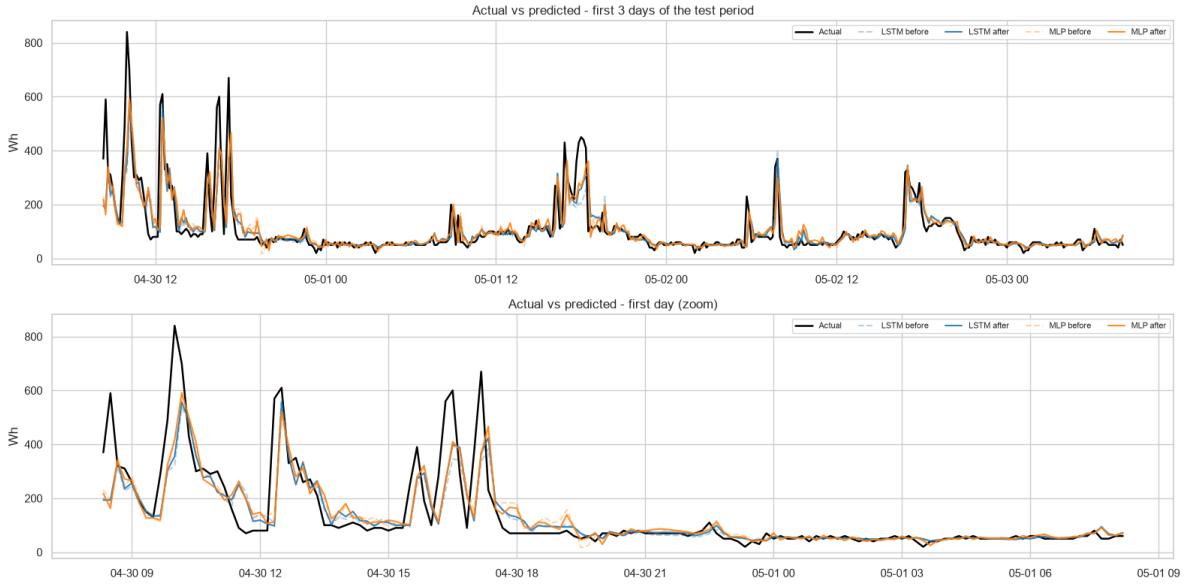


Figure 11: Actual and predicted consumption in the first three days (top) and the first day (bottom) of the test period, before and after optimization.

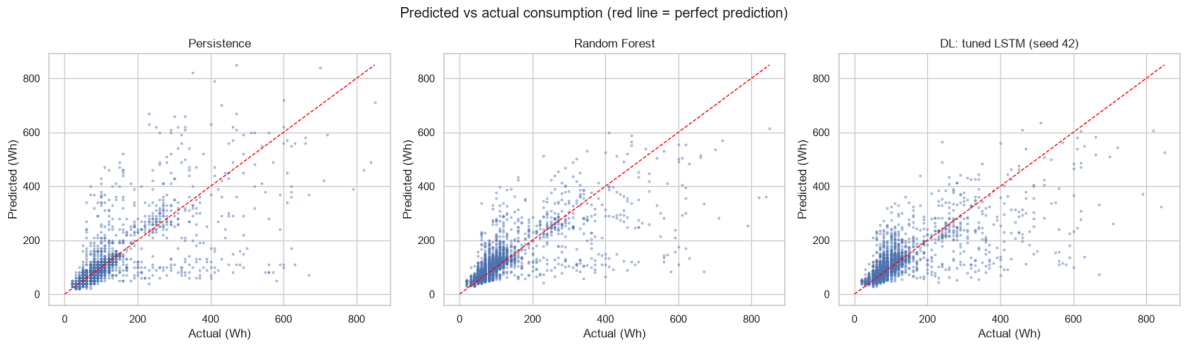


Figure 12: Predicted against actual consumption on the test set (the red line marks a perfect prediction).

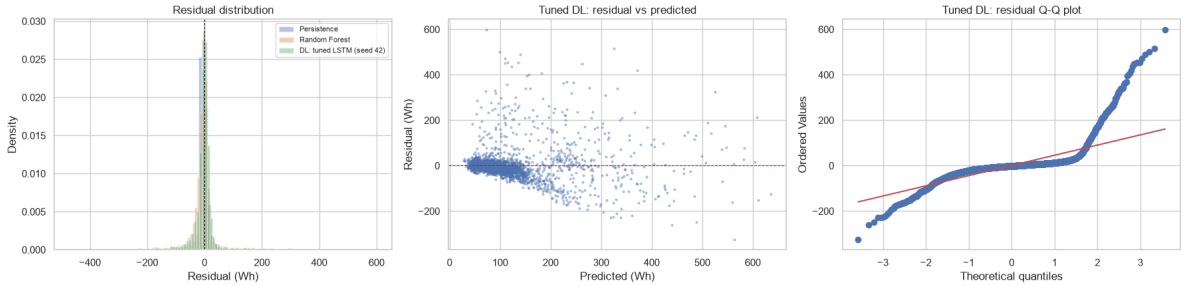


Figure 13: Residual distribution (left), residuals against predictions (centre) and normal Q-Q plot (right) of the tuned LSTM.

MODEL OPTIMIZATION

a) Scope and a reliable way to rank configurations

The optimization concentrates on the two light-weight candidates found above: the **single-layer LSTM** and a **single-hidden-layer MLP**. The 14 features, the 80 % / 20 % split and the rule that the test set is used only once are unchanged. No further data exist, so reliability had to come from the evaluation method. The Results section showed that the differences between models are of the same size as the random-seed noise; a single validation run cannot rank them. Four techniques, all using only the training part, were applied:

- **Walk-forward cross-validation** [9] inside the training part: each candidate is trained and scored on four consecutive time blocks (expanding window, scalers re-fitted per fold, early-stopping data taken from the fold's own training rows), so it must work in different periods and not only in the last validation weeks;
- **several random seeds** to average out the run-to-run variation of neural-network training;
- a **two-stage search**: cheap single-split screening of many configurations, followed by walk-forward re-ranking of the shortlist only;
- a **paired block bootstrap** [10] on the final test comparison: whole days of test errors are resampled, which gives a confidence interval for every difference between models without enlarging the test set.

b) Hyper-parameter tuning

The tuned hyper-parameters are the learning rate, the batch size, the number of neurons per layer, the size of the optional dense head, the dropout rate and the look-back window (Table 11). The number of layers is fixed at one. The number of epochs is not a free parameter: with early stopping the maximum was raised from 40 to 100 and the networks stopped after about 12–93 epochs, so the cap never limited training. **Random search** [6] was used first because, with six parameters, random sampling covers every range better than a grid of the same cost (16 LSTM and 20 MLP configurations, each screened on the validation set). A **grid search** over learning rate × batch size around the best random configuration followed. The shortlist (the start configuration, the best random and the best grid configuration) was then re-ranked with the walk-forward cross-validation, and a tuned configuration was accepted only if it was at least 0.5 % better than the start.

Table 11: Hyper-parameter search spaces.

Hyper-parameter	LSTM search space	MLP search space
Neurons (units)	16, 32, 64, 96, 128	32, 64, 128, 256, 512
Dense head	0, 16, 32, 64	none
Dropout rate	0.1, 0.2, 0.3, 0.4, 0.5	0.1, 0.2, 0.3, 0.4, 0.5
Learning rate	3e-4, 5e-4, 1e-3, 2e-3, 3e-3	3e-4, 5e-4, 1e-3, 2e-3, 3e-3
Batch size	32, 64, 128, 256	32, 64, 128, 256
Look-back (steps)	6, 12, 24	3, 6, 12, 24
Layers / epochs	1 layer; early stopping, cap 100	1 hidden layer; early stopping, cap 100

LSTM. The 16 random configurations reached 58.78–61.23 Wh validation RMSE; none beat the start (58.70 Wh). The grid search found one run at 57.68 Wh, which looked like a 1.7 % gain on the single validation split. The walk-forward test told a different story (Table 12): the best candidate is only 0.08 % better than the start, far below the 0.5 % threshold, and needs 3.4 times more parameters. The 57.68 Wh result was largely the luck of one run, which is exactly what the reliable procedure is meant to expose. The LSTM therefore keeps its hyper-parameters (64 units, dense 32, learning rate 0.001, batch 128, look-back 12): the optimum is flat and it was already close to it. **MLP.** The best random configuration (128 units, dropout 0.1, learning rate 0.001, batch 256, look-back 6) is 1.8 % better than the start in the walk-forward test and better in all four folds, and it has half the parameters (11 k against 21.8 k) because

the shorter look-back means fewer inputs. The grid refinement did not improve on it, and it was accepted.

Table 12: Walk-forward cross-validation (training part, two seeds): start configuration against the final optimized one.

Model	Version	Parameters	Walk-forward RMSE (Wh)	Fold 1	Fold 2	Fold 3	Fold 4
LSTM	before (start)	22,337	65.76	69.7	71.1	61.5	60.8
LSTM	after (optimized)	22,337	65.69	69.8	71.3	61.1	60.6
MLP	before (start)	21,761	66.66	69.6	72.8	62.6	61.6
MLP	after (optimized)	11,009	65.47	67.9	71.9	61.3	60.8

The absolute walk-forward RMSE (about 65 Wh) is higher than the validation RMSE (58–60 Wh) because the first folds cover January and February and have little training data (fold 1 trains on about 3,100 rows); configurations must be compared with each other, not with the validation numbers. The last two folds, with more training data, are at about 61 Wh.

c) Regularization: dropout layers and dropout rates

Dropout [5] switches off a share of the units in every training step, so the network cannot depend on single units. Rates between 0 and 0.5 were compared on the tuned configurations, each rate with two seeds (Figure 14). For the LSTM, the mean validation RMSE was 59.34 Wh without dropout (training RMSE 55.9 Wh, a gap of +3.5 Wh, i.e. overfitting), 59.18 at 0.1, **58.55 at 0.2**, 58.71 at 0.3, 58.98 at 0.4 and 58.71 at 0.5: rates of 0.2–0.5 are within the seed spread, whereas no dropout is clearly worse. For the MLP the curve is flat between 0.1 and 0.3 (60.35, 60.40, 60.38 Wh) and worse without dropout (60.75 Wh); 0.1 was kept. High rates start to under-fit (the MLP training error rises to 62.5 Wh) while the MAE keeps falling, because the network shrinks toward the typical consumption and under-predicts peaks. The chosen rates are 0.2 (LSTM) and 0.1 (MLP).

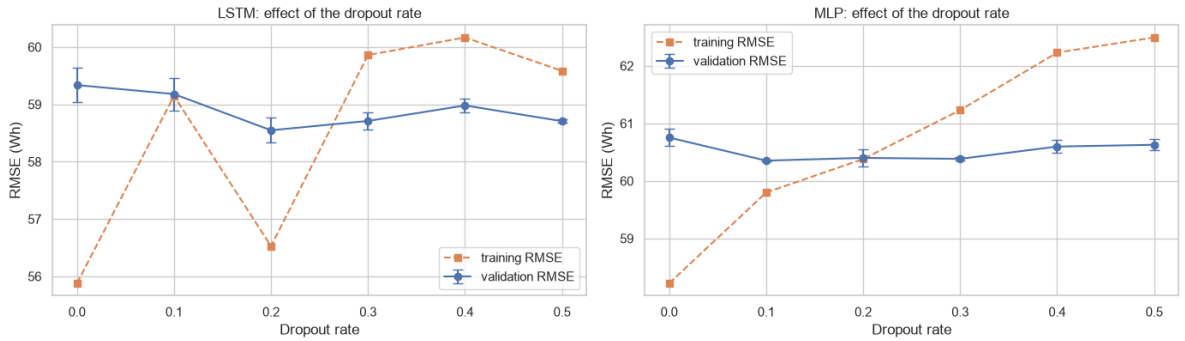


Figure 14: Effect of the dropout rate on the validation and training RMSE (mean ± standard deviation over two seeds).

d) Early stopping

Early stopping [11] watches the validation loss after every epoch and stops when it has not improved for *patience* epochs; the weights of the best epoch are restored, so training for too long cannot hurt. Patience values of 3, 6, 10 and 15 were compared with two seeds (Figure 15). For the LSTM, the validation RMSE was 58.69, 58.55, 58.62 and 58.78 Wh with 20, 34, 36 and 42 epochs on average: all within 0.4 %, so the smallest patience (3) was chosen. It gives the same accuracy with about 40 % fewer epochs (15 s instead of 25 s per run). For the MLP, patience 3 stops too early (60.82 Wh, +0.8 % worse) because its validation loss has temporary plateaus; patience 6 (60.35 Wh) is as good as 10 and 15 and was kept.

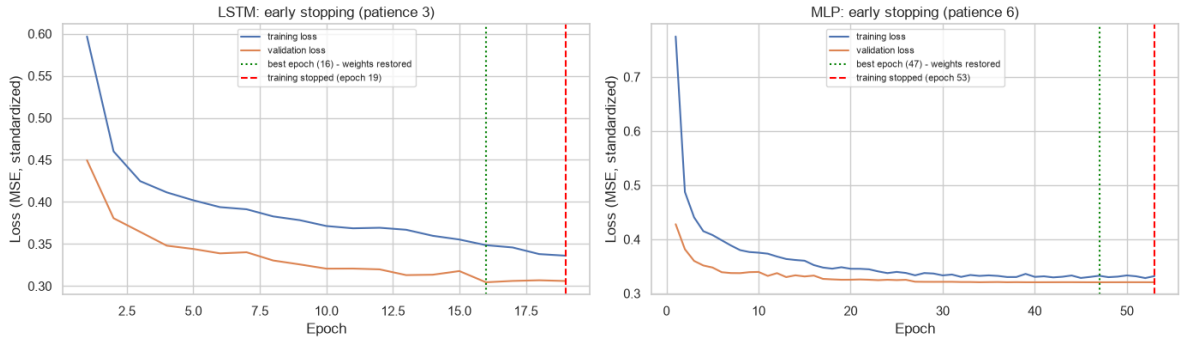


Figure 15: Training and validation loss with the best epoch (weights restored) and the stopping point for the LSTM (left) and the MLP (right).

e) Final optimized models and seed ensemble

The final configurations are: **LSTM** - 64 units, dense head 32, dropout 0.2, learning rate 0.001, batch 128, look-back 12, patience 3, maximum 100 epochs; **MLP** - one hidden layer of 128 units, dropout 0.1, learning rate 0.001, batch 256, look-back 6, patience 6, maximum 100 epochs. Before accepting them, they were compared once more with the walk-forward test (Table 12): the optimization never replaced a configuration by a worse one. Neural-network results vary from run to run, and averaging the predictions of networks trained with different seeds reduces this variance without any additional data. A three-seed ensemble was adopted for both families because it lowered the *validation* RMSE (LSTM 59.01 to 58.69 Wh, MLP 60.46 to 60.12 Wh); the decision did not use the test set.

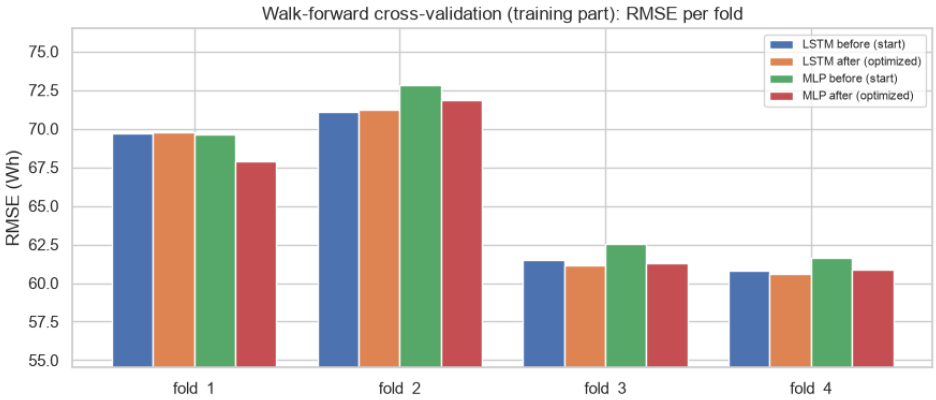


Figure 16: Walk-forward cross-validation: RMSE of the start and the optimized configurations in each fold.

f) Improvement in performance metrics

Table 13 compares the models before and after optimization on the test set (the same 3,947 intervals as before; deep-learning rows are the mean of three seeds, and the ensemble averages the three networks). Table 14 gives the 95 % confidence interval of the RMSE difference from the paired block bootstrap (2,000 resamples of whole days).

Table 13: Test-set results before and after optimization.

Model	MAE (Wh)	RMSE (Wh)	MAPE (%)	R^2
Persistence	26.65	66.44	21.9	0.467
Linear Regression	27.13	59.82	25.1	0.568
Random Forest	26.33	58.40	23.9	0.588
LSTM before optimization (mean of 3 seeds)	26.24	58.20	23.4	0.591
LSTM before optimization (3-seed ensemble)	25.67	57.39	22.8	0.603
LSTM after optimization (mean of 3 seeds)	25.78	57.23	23.2	0.605
LSTM after optimization (3-seed ensemble)	**25.45**	**56.91**	**22.8**	**0.609**
MLP before optimization (mean of 3 seeds)	27.40	58.11	25.5	0.593
MLP before optimization (3-seed ensemble)	27.09	57.88	25.1	0.596
MLP after optimization (mean of 3 seeds)	27.38	57.95	25.6	0.595
MLP after optimization (3-seed ensemble)	26.95	57.64	25.1	0.599

Table 14: Paired block bootstrap of the test RMSE difference (negative = model A has the lower error).

Model A	compared with B	RMSE(A) − RMSE(B) (Wh)	95 % interval	Verdict
LSTM after (ensemble)	LSTM before	-0.32	-0.94 to 0.32	not separable
LSTM after (ensemble)	Persistence	-9.48	-11.39 to -7.50	A is better
LSTM after (ensemble)	Linear Regression	-3.07	-4.21 to -1.90	A is better
LSTM after (ensemble)	Random Forest	-1.49	-2.67 to -0.42	A is better
MLP after (ensemble)	MLP before	-0.39	-0.72 to -0.07	A is better
MLP after (ensemble)	Persistence	-8.73	-10.63 to -6.82	A is better
MLP after (ensemble)	Linear Regression	-2.32	-3.52 to -1.22	A is better
MLP after (ensemble)	Random Forest	-0.73	-1.82 to 0.37	not separable
LSTM after	MLP after	-0.75	-1.59 to 0.02	not separable

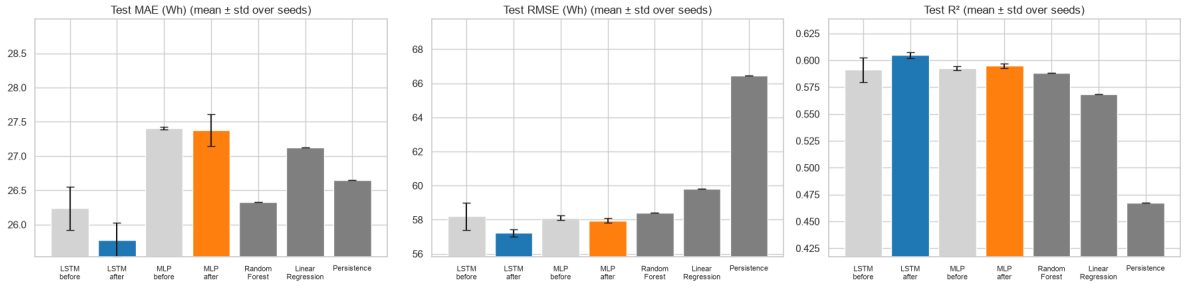


Figure 17: Test MAE, RMSE and R^2 before and after optimization (mean ± standard deviation over three seeds), with the baselines.

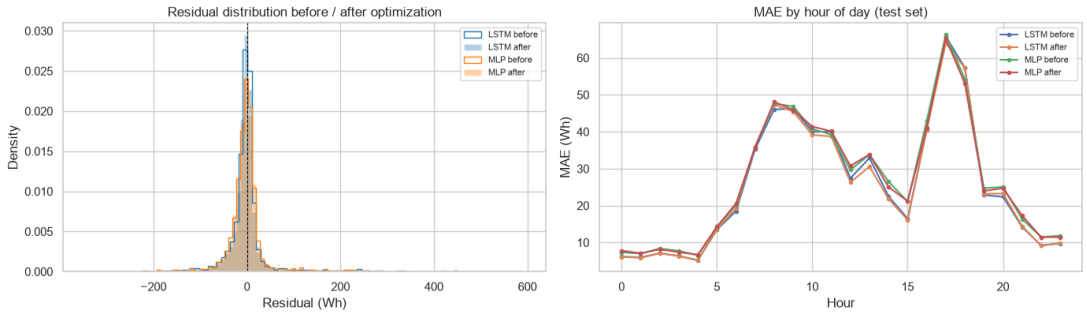


Figure 18: Residual distribution (left) and MAE by hour of the day (right) before and after optimization.

Discussion. The optimization produced small gains, and most of them cannot be separated from random variation. The LSTM test RMSE improved by 1.7 % (58.20 to 57.23 Wh; all three seeds improved and

the spread between seeds fell from 0.81 to 0.21 Wh), but its hyper-parameters did not change; only the patience did. The walk-forward test, the more reliable measure, shows only a 0.1 % difference, and the bootstrap cannot separate the ensembles before and after (−0.32 Wh, interval −0.94 to +0.32). A part of the 1.7 % is therefore seed luck: the 'before' average is pulled up by one weak seed. The honest summary is that the LSTM was already close to its optimum, that the optimization confirmed this and shortened training by about 40 %, and that the ensemble lowers the variance. The MLP profited more in the reliable measure (−1.8 % in the walk-forward test, half the size), but only slightly on the test set (−0.3 %).

Final ranking. The optimized LSTM ensemble (RMSE 56.91 Wh, MAE 25.45 Wh, R^2 0.609) is better than persistence (difference −9.5 Wh, interval −11.4 to −7.5), linear regression (−3.1 Wh, −4.2 to −1.9) and, importantly, the **random forest (−1.5 Wh, −2.7 to −0.4)**, a difference that could not be established before the optimization. The optimized MLP ensemble beats persistence and linear regression but cannot be separated from the random forest or from the LSTM. For an edge device the 11 k-parameter MLP, which trains in 5–9 seconds, is the cheaper choice at almost the same accuracy; the LSTM gives the lowest error. A single network is preferable to the ensemble when the threefold inference cost is not acceptable.

CHALLENGES AND SOLUTIONS

Table 15 lists the main difficulties met during the work and how each was addressed. Several of them are errors of an earlier version of the experiment that were found by reviewing the whole pipeline; fixing them changed the conclusions more than any model choice.

Table 15: Main challenges and the solutions applied.

Challenge	Why it mattered	Solution
The sequence windows ended one row before the target	the network could not see the previous interval's consumption, which persistence uses	windows end at the target row; automatic alignment test
Baselines and networks saw different information	sensor readings of the same interval would not exist in a real forecast; comparison was unfair	sensors shifted by one interval; leakage test; identical inputs for all models
Models were compared on the test set	test results influenced the design; optimistic estimates	validation set carved from the training part; test used once
Feature selection was computed but not applied	noise columns (rv1, rv2) were fed to the models	voting of three methods; duplicates removed; selected set used
Seasonal drift of the sensors	a random forest using raw sensors scored 61.5 Wh on validation but 77.2 Wh on the test weeks	walk-forward feature-group test; environment columns dropped (test RMSE 58.4 Wh)
Small differences, noisy training	single runs gave misleading rankings (the 57.68 Wh grid-search result)	several seeds, walk-forward cross-validation, acceptance threshold, block bootstrap
Spiky, partly unpredictable target	RMSE is dominated by bursts of 400–600 Wh that no model anticipates	RMSE, MAE and MAPE reported together; loss functions compared; limits stated
Only 4.5 months of data, split fixed at 80 / 20	no information about other seasons; limited test period	reliability increased by the evaluation method; limits stated in the conclusion
Reproducibility and environment	the notebook read data only from Google Drive	fixed seeds, local-path fallback, reusable code in src/

CONCLUSION

The report developed and optimized deep-learning models that forecast the appliance energy consumption of the next 10 minutes. The data are complete but spiky, with a daily evening peak and a strong dependence on the immediately preceding consumption. A leakage-free formulation, features built from past consumption and the time of day, and a feature selection that is robust to seasonal drift produced a compact input of 14 features. Baselines, six deep architectures, and the choices of activation function, optimizer, loss and regularization were compared on a validation set that was taken from the training part; the test set was used only for the final comparison.

The best model, an ensemble of three single-layer LSTM networks, reaches an RMSE of 56.9 Wh, an MAE of 25.4 Wh and an R^2 of 0.609 on the test set, about 14 % lower RMSE than persistence (66.4 Wh) and 2.6 % lower than the random forest (58.4 Wh); both differences are statistically supported. The optimization, which used random and grid search, dropout, early stopping, walk-forward cross-validation and seed ensembling, showed that the networks were already near a flat optimum: the gains are small (about 1–2 %), the MLP became half as large at the same accuracy, and training became about 40 % shorter. Deep learning gives only a modest advantage over a well-built tree model on this data set, and a small MLP is a good choice for an edge device.

Potential areas for future work

- **More data** across all seasons and several houses, to evaluate and train for seasonal changes, which the 4.5 months cannot cover;
- **additional information** about the behaviour of the occupants (occupancy, appliance-level or calendar information), because the remaining error is dominated by unpredictable bursts;
- **peak-aware or probabilistic forecasting** (quantile loss, prediction intervals) instead of a single mean forecast;
- **Bayesian optimization** and a larger search budget, although the flat optimum found here suggests small gains;
- **edge deployment**: model compression (quantization, TensorFlow Lite), latency and memory measurements, and online adaptation to drift.

REFERENCES

[1] “Appliances Energy Prediction,” UCI Machine Learning Repository. [Online]. Available: https://archive.ics.uci.edu/dataset/374/appliances+energy+prediction

[2] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[3] K. Cho *et al.*, “Learning phrase representations using RNN encoder–decoder for statistical machine translation,” in *Proc. EMNLP*, 2014, pp. 1724–1734.

[4] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. ICLR*, 2015.

[5] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *J. Mach. Learn. Res.*, vol. 15, pp. 1929–1958, 2014.

[6] J. Bergstra and Y. Bengio, “Random search for hyper-parameter optimization,” *J. Mach. Learn. Res.*, vol. 13, pp. 281–305, 2012.

[7] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[8] I. Guyon, J. Weston, S. Barnhill, and V. Vapnik, “Gene selection for cancer classification using support vector machines,” *Machine Learning*, vol. 46, pp. 389–422, 2002.

[9] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. OTexts, 2021. [Online]. Available: https://otexts.com/fpp3/

[10] H. R. Künsch, “The jackknife and the bootstrap for general stationary observations,” *Annals of Statistics*, vol. 17, no. 3, pp. 1217–1241, 1989.

[11] L. Prechelt, “Early stopping – but when?” in *Neural Networks: Tricks of the Trade*. Springer, 1998, pp. 55–69.

[12] P. J. Huber, “Robust estimation of a location parameter,” *Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, 1964.

[13] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[14] M. Abadi *et al.*, “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015. [Online]. Available: https://www.tensorflow.org